Sistema de Gestión de Incidencias e Infraestructura

Documentación Técnica del Proyecto

Ciclo Formativo de Grado Superior — ASIR

Administración de Sistemas Informáticos en Red

Índice

1. Introducción 4
1.1 Tecnologías Utilizadas 4
2. Objetivos del Proyecto 5
2.1 Roles de Usuario 5
3. Arquitectura de Red 6
3.1 Red DMZ — 192.168.50.0/24 6
3.2 Red Interna — 192.168.60.0/24 7
3.3 Flujo de una Petición Web 7
4. Infraestructura Virtual (Vagrant) 7
4.1 ¿Porque Vagrant? 7
4.2 Máquinas Virtuales 7
4.3 Vagrantfile — Configuración Clave 8
4.4 Rutas entre Redes 8
4.5 Script de Backend (backend.sh) 9
5. Configuración del Firewall 9
5.1 Activar IP Forwarding 9
5.2 Políticas por Defecto 10
5.3 Reglas de INPUT 10
5.4 Reglas de FORWARD — Tráfico entre Redes 10
5.5 NAT — Salida a Internet 11
5.6 Guardar Reglas de Forma Permanente 11
5.7 Resumen de Reglas 11
6. Base de Datos (MariaDB) 12
6.1 Configuración de MariaDB 12
6.2 Estructura de la Base de Datos 12
6.3 Modelo Entidad-Relación 12
6.4 Modelo Entidad-Relación 14
6.5 Permisos de Usuario 20
7. Aplicación Web 20
7.1 Configuración de Nginx (Balanceador) 20
7.2 Sistema de Autenticación 21
7.3 Conexión a la Base de Datos 22
8. Windows Server — Active Directory 22
8.1 Instalación y Configuración del Dominio 23
8.2 Unidades Organizativas 24
8.3 Creación de Usuarios en AD 24
9. Windows Server — DNS 25
9.1 Instalación y Zonas DNS 25
9.2 Registros DNS 26
9.3 Registros Creados 26
10. Windows Server — DHCP 27
10.1 Configuración del Scope DHCP 27

10.2 Reservas para Servidores 27
11. Windows Server — FTP 28
11.1 Instalación del Servicio FTP 28
12. Sistema de Monitorización (Ampliación) 29
12.1 Arquitectura del Sistema 29
12.2 Servidor Flask (servidor_flask.py) 29
12.3 Script de Consulta (admin_servidor.py) 30
12.4 Consulta desde PHP 31
12.5 Arranque Automático del Flask 31
13. Usuarios y Accesos 32
13.1 Usuarios del Sistema Web 32
13.2 Acceso a la Aplicación 33
14. Solución de Errores 33
15. Herramientas Utilizadas 34
16. Conclusiones 35

1. Introducción

Verificanet es un sistema integral de gestión de incidencias e infraestructura diseñado para empresas del sector TI. El proyecto ha sido desarrollado como trabajo final del Ciclo Formativo de Grado Superior de ASIR, integrando conocimientos de administración de sistemas, redes, bases de datos y desarrollo web.

El sistema permite a los empleados de diferentes departamentos gestionar incidencias técnicas, consultar documentación y supervisar el estado de los servidores en tiempo real, todo desde una interfaz web centralizada y accesible.

La infraestructura está basada en máquinas virtuales gestionadas con Vagrant y VirtualBox, con una arquitectura en capas que separa la zona desmilitarizada (DMZ) de la red interna, garantizando la seguridad y el aislamiento de los servicios críticos. El Windows Server integra servicios empresariales como Active Directory, DNS, DHCP y FTP, y actúa además como intermediario del sistema de monitorización.

1.1 Tecnologías Utilizadas

Tecnología	Versión	Uso
Vagrant + VirtualBox	2.4 / 7.x	Gestión y virtualización de VMs
Ubuntu 22.04 LTS	Jammy	SO de las VMs Linux
Windows Server 2019	2019	Servidor Windows empresarial
Nginx	1.18	Balanceador de carga HTTP
Apache + PHP	2.4 / 8.x	Backend de la aplicación web
MariaDB	10.6	Base de datos relacional
Python 3.12 + Flask	3.12	Scripts de monitorización y API REST
iptables	-	Firewall y NAT
Active Directory	-	Gestión centralizada de usuarios
OpenSSH	-	Acceso remoto y monitorización SSH

2. Objetivos del Proyecto

El proyecto Verificanet persigue los siguientes objetivos principales:

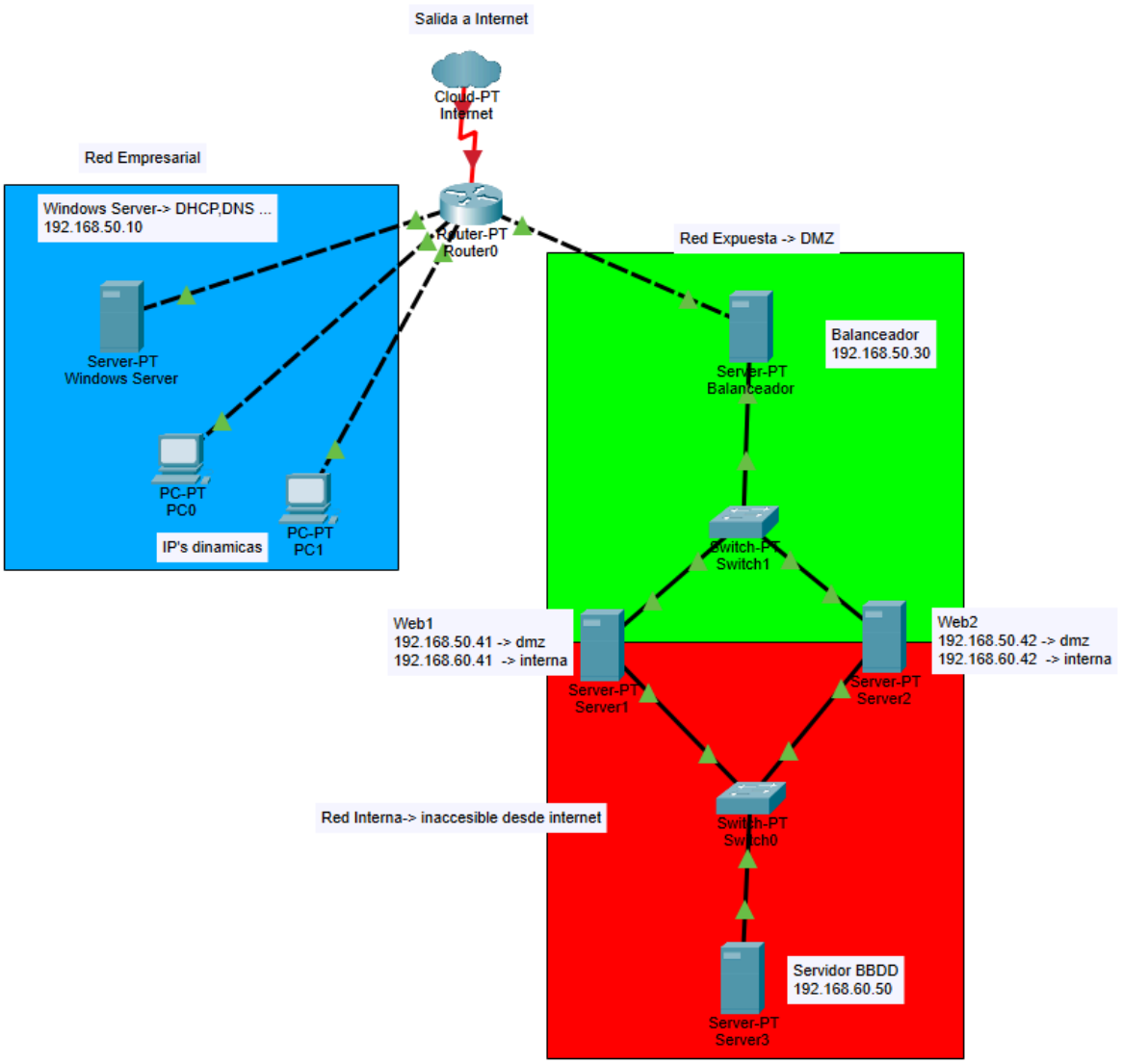
- Diseñar e implementar una infraestructura de red segmentada con DMZ y red interna separadas por un firewall.
- Desplegar una aplicación web de gestión de incidencias con múltiples roles de usuario y dashboards diferenciados por departamento.
- Implementar un balanceador de carga con alta disponibilidad mediante dos backends Apache en paralelo.
- Proteger la base de datos en una red interna separada, accesible solo desde los backends.
- Integrar un Windows Server con Active Directory, DNS, DHCP y FTP que proporcione servicios empresariales reales.
- Desarrollar como ampliación un sistema de monitorización de infraestructura en tiempo real accesible desde la propia web.
- Automatizar el despliegue completo mediante scripts de aprovisionamiento bash y PowerShell.

2.1 Roles de Usuario

Rol	Página	Funcionalidades
Admin	admin.php	Gestión completa, monitorización, asignación de incidencias
Empleado	empleado.php	Ver y gestionar incidencias asignadas, base de conocimiento
Ventas	ventas.php	Gestión de clientes, contratos y servicios
Administración	administracion.php	Gestión administrativa, empleados
Auditoría	auditoria.php	Logs, incidencias cerradas, calendario de reuniones
Técnico	tecnico.php	Resolución técnica de incidencias asignadas
Cliente	servicios.php	Crear incidencias, ver contratos y servicios contratados

3. Arquitectura de Red

La infraestructura de Verificanet está dividida en dos redes diferenciadas separadas por un firewall que controla todo el tráfico entre ellas. Esta separación garantiza que los servicios críticos como la base de datos no sean accesibles directamente desde el exterior.



3.1 Red DMZ — 192.168.50.0/24

La DMZ (Zona Desmilitarizada) aloja los servicios accesibles desde el exterior y es la red principal del proyecto:

Nombre	IP	Función
router	192.168.50.1	Enrutamiento y salida a Internet
firewall	192.168.50.20	Filtrado de tráfico entre DMZ y red interna
web (nginx)	192.168.50.30	Balanceador de carga HTTP con ip_hash
backend1 (apache)	192.168.50.41	Servidor PHP — Aplicación web

backend2 (apache)	192.168.50.42	Servidor PHP — Aplicación web (HA)
Windows Server	192.168.50.10	AD, DNS, DHCP, FTP, Flask API

3.2 Red Interna — 192.168.60.0/24

La red interna aloja los servicios críticos que nunca deben estar expuestos directamente. Solo el firewall tiene acceso a ambas redes:

Nombre	IP	Función
firewall	192.168.60.20	Gateway hacia la DMZ
database (MariaDB)	192.168.60.50	Base de datos del sistema — solo accesible desde backends

3.3 Flujo de una Petición Web

El recorrido completo de una petición desde el navegador hasta la base de datos es el siguiente:

1. El usuario accede a localhost:8080 desde su navegador en el PC host.
2. Nginx (192.168.50.30) recibe la petición y la reenvía a backend1 o backend2 usando ip_hash para mantener la sesión.
3. El backend PHP procesa la petición y conecta a MariaDB (192.168.60.50) pasando por el firewall.
4. El firewall valida que la conexión está permitida (regla FORWARD puerto 3306) y la reenvía a la red interna.
5. MariaDB procesa la consulta y devuelve los datos al backend.
6. El backend genera el HTML de respuesta y lo devuelve al usuario a través de Nginx.

4. Infraestructura Virtual (Vagrant)

Toda la infraestructura Linux se despliega de forma automatizada usando Vagrant con VirtualBox. El Vagrantfile define 6 máquinas virtuales que se aprovisionan automáticamente con scripts bash al ejecutar vagrant up.

4.1 ¿Porque Vagrant?

Vagrant es una herramienta de virtualización muy útil ya que te da la posibilidad de iniciar varias máquinas virtuales a la vez, con la misma configuración es decir mismo Sistema Operativo, versión, portabilidad, Entorno desechables …

4.2 Máquinas Virtuales

VM	RAM	Red	Script de aprovisionamiento
router	512 MB	DMZ + Interna	router.sh
firewall	512 MB	DMZ + Interna	firewall.sh
web	512 MB	DMZ	web-frontend.sh + Rutas_dmz.sh

backend1	512 MB	DMZ	backend.sh + Rutas_dmz.sh
backend2	512 MB	DMZ	backend.sh + Rutas_dmz.sh
database	1024 MB	Interna	database.sh + Rutas_interna.sh

4.3 Vagrantfile — Configuración Clave

La configuración más importante del Vagrantfile incluye el boot_timeout de 600 segundos (para evitar timeouts con redes intnet) y la configuración del tipo de tarjeta de red:

```
Vagrant.configure('2') do |config|
  config.vm.box = 'ubuntu/jammy64'
  config.vm.boot_timeout = 600
  config.ssh.insert_key = false

  config.vm.define 'firewall' do |firewall|
    firewall.vm.hostname = 'firewall'
    firewall.vm.network 'private_network',
      ip: '192.168.50.20', virtualbox__intnet: 'dmz'
    firewall.vm.network 'private_network',
      ip: '192.168.60.20', virtualbox__intnet: 'interna'
    firewall.vm.provider 'virtualbox' do |vb|
      vb.name = 'verificanet-firewall'
      vb.memory = 512
      vb.customize ['modifyvm', :id, '--nictype2', '82540EM']
    end
    firewall.vm.provision 'shell', path: 'provision/firewall.sh'
  end
end
```

4.4 Rutas entre Redes

Las máquinas en la DMZ necesitan una ruta estática para llegar a la red interna. El script Rutas_dmz.sh configura esta ruta apuntando al firewall como gateway:

```
#!/bin/bash
# Rutas_dmz.sh — Configura ruta hacia la red interna

ip route del 192.168.60.0/24 2>/dev/null
ip route add 192.168.60.0/24 via 192.168.50.20

cat > /etc/netplan/99-custom-routes.yaml <<EOF
network:
  version: 2
  ethernets:
    enp0s8:
      routes:
        - to: 192.168.60.0/24
          via: 192.168.50.20
EOF
```

```
netplan apply
```

De forma análoga, el script Rutas_interna.sh configura la ruta inversa en la base de datos hacia la DMZ via 192.168.60.20.

4.5 Script de Backend (backend.sh)

El script de aprovisionamiento de los backends instala Apache y PHP, copia los archivos de la aplicación web desde la carpeta compartida y aplica los permisos correctos:

```
#!/bin/bash
apt-get update
apt-get install -y apache2 php php-mysql

# Limpiar directorio web
rm -rf /var/www/html/*

# Copiar archivos del proyecto
if [ -d '/vagrant/html-backup' ]; then
    cp -r /vagrant/html-backup/. /var/www/html/
fi

# Eliminar index.html para que Apache use index.php
rm -f /var/www/html/index.html

# Permisos
chown -R www-data:www-data /var/www/html
chmod -R 755 /var/www/html

systemctl enable apache2
systemctl restart apache2
```

5. Configuración del Firewall

El firewall es la máquina central de seguridad del proyecto. Tiene dos interfaces de red: enp0s8 en la DMZ (192.168.50.20) y enp0s9 en la red interna (192.168.60.20). Gestiona todo el tráfico entre ambas redes mediante reglas iptables.

5.1 Activar IP Forwarding

El primer paso es activar el reenvío de paquetes entre interfaces de forma permanente:

```
# Activar ip_forward en tiempo real
sysctl -w net.ipv4.ip_forward=1

# Hacerlo permanente en sysctl.conf
sed -i '/net.ipv4.ip_forward/d' /etc/sysctl.conf
echo 'net.ipv4.ip_forward=1' >> /etc/sysctl.conf
```

5.2 Políticas por Defecto

Se establece una política restrictiva: todo el tráfico está denegado por defecto y solo se permite lo que se especifica explícitamente:

```
# Limpiar reglas anteriores
iptables -F
iptables -X
iptables -t nat -F

# Políticas por defecto — todo denegado
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT
```

5.3 Reglas de INPUT

Se permiten las conexiones ya establecidas y el acceso SSH de gestión:

```
# Permitir tráfico de conexiones ya establecidas
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Permitir SSH para gestión del firewall
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

5.4 Reglas de FORWARD — Tráfico entre Redes

Las reglas de FORWARD son las más importantes. Controlan qué tráfico puede pasar de una red a otra:

```
# Tráfico ya establecido puede seguir pasando
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# Todo el tráfico entre DMZ (enp0s8) y red interna (enp0s9)
iptables -A FORWARD -i enp0s8 -o enp0s9 -j ACCEPT
iptables -A FORWARD -i enp0s9 -o enp0s8 -j ACCEPT

# Balanceador puede llegar a los backends por HTTP
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.41 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 192.168.50.30 -d 192.168.50.42 -p tcp --dport 80 -j ACCEPT

# Los backends pueden conectar a la base de datos
iptables -A FORWARD -s 192.168.50.41 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT
iptables -A FORWARD -s 192.168.50.42 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT

# El Windows Server puede consultar la base de datos
iptables -A FORWARD -s 192.168.50.10 -d 192.168.60.50 -p tcp --dport 3306 -j ACCEPT
```

5.5 NAT — Salida a Internet

Se configura NAT (MASQUERADE) para que las máquinas de la DMZ y la red interna puedan salir a Internet a través de la interfaz NAT (enp0s3) del firewall:

```
# NAT para salida a Internet
iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE

# Permitir salida a Internet desde la DMZ
iptables -A FORWARD -s 192.168.50.0/24 -i enp0s8 -o enp0s3 -j ACCEPT

# Permitir salida a Internet desde la red interna
iptables -A FORWARD -s 192.168.60.0/24 -i enp0s9 -o enp0s3 -j ACCEPT
```

5.6 Guardar Reglas de Forma Permanente

Para que las reglas sobrevivan a los reinicios del sistema se usa iptables-persistent:

```
# Instalar iptables-persistent
apt-get install -y iptables iptables-persistent

# Guardar las reglas actuales
iptables-save > /etc/iptables/rules.v4
```

5.7 Resumen de Reglas

Origen	Destino	Descripción
Cualquiera	Cualquiera	ESTABLISHED/RELATED — tráfico ya iniciado
Cualquiera	Firewall:22	SSH de gestión del firewall
enp0s8 (DMZ)	enp0s9 (Interna)	Todo el tráfico DMZ → Red Interna
enp0s9 (Interna)	enp0s8 (DMZ)	Todo el tráfico Interna → DMZ
192.168.50.30	50.41/42:80	Nginx → Backends HTTP
192.168.50.41/42	60.50:3306	Backends → MariaDB
192.168.50.10	60.50:3306	Windows Server → MariaDB
DMZ / Interna	Internet	Salida a Internet (NAT MASQUERADE)

6. Base de Datos (MariaDB)

La base de datos MariaDB está instalada en la máquina database (192.168.60.50) dentro de la red interna. Está configurada para escuchar en todas las interfaces pero el firewall garantiza que solo los backends y el Windows Server puedan conectarse.

6.1 Configuración de MariaDB

La configuración más importante es cambiar el bind-address para que MariaDB acepte conexiones remotas:

```
# Cambiar bind-address en /etc/mysql/mariadb.conf.d/50-server.cnf
sed -i 's/^bind-address.*/bind-address = 0.0.0.0/' \
    /etc/mysql/mariadb.conf.d/50-server.cnf

systemctl restart mariadb

# Verificar que escucha en todas las interfaces
ss -tlnp | grep 3306
# Resultado esperado: LISTEN 0  80  0.0.0.0:3306  0.0.0.0:*
```

6.2 Estructura de la Base de Datos

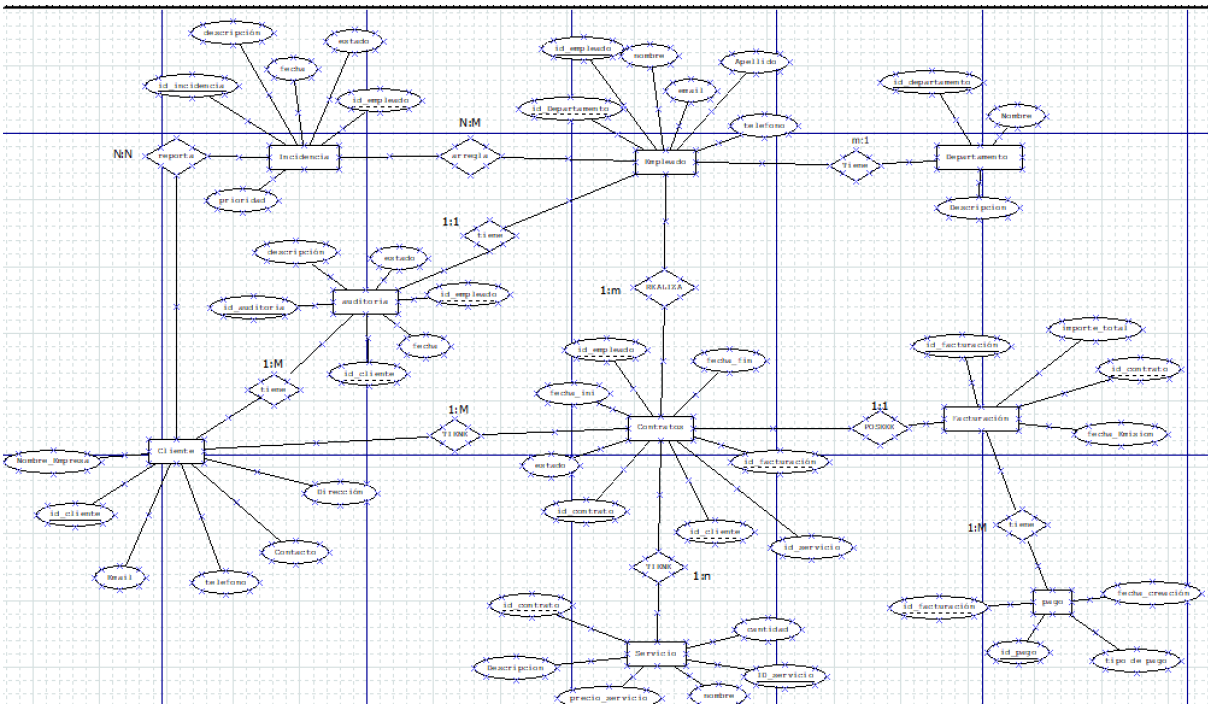
La base de datos verificanet_servicios contiene las siguientes tablas:

Tabla	Descripción
departamentos	Departamentos: Soporte, Ventas, Administración, Sistemas, Auditoría
empleados	Personal de la empresa con su departamento asignado
clientes	Clientes particulares y empresas con sus datos de contacto
servicios	Catálogo de servicios: soporte, mantenimiento, desarrollo, consultoría
contratos	Contratos activos entre clientes y servicios
incidencias	Tickets de soporte con estado, prioridad y empleado asignado
usuarios	Cuentas de acceso al sistema con rol y contraseña hasheada
kb_articulos	Base de conocimiento con soluciones a problemas comunes

6.3 Modelo Entidad-Relación

En este apartado veremos la gestión de la Base de Datos y presentando el Modelo Entidad-Relación.
Posteriormente se podrá ver de forma gráfica las principales tablas, entidades y atributos de lo que pasará a ser nuestra Base de Datos.
Además está normalizada en la 3º Forma.
El objetivo principal del ER es dar a conocer un poco la estructura de lo que será la empresa, identificando las claves primarias, secundarias y dependencias …

Que pasarán a ser parte del modelo lógico.



Modelo relacional. A partir del Modelo Entidad-Relación, se obtiene el siguiente modelo relacional, donde se especifican las tablas de la base de datos junto con sus claves primarias (PK), claves foráneas (FK) y atributos correspondientes:

- departamentos: id_departamento (PK), nombre, descripcion
- empleados: id_empleado (PK), id_departamento (FK), nombre, apellido, email, telefono, fecha_contratacion
- clientes: id_cliente (PK), nombre, apellido, nombre_empresa, email, telefono, direccion, tipo_cliente, fecha_registro
- servicios: id_servicio (PK), nombre, descripcion, precio_base, tipo_servicio
- contratos: id_contrato (PK), id_cliente (FK), id_servicio (FK), fecha_inicio, fecha_fin, estado, notas
- incidencias: id_incidencia (PK), id_cliente (FK), id_empleado_asignado (FK), titulo, descripcion, estado, prioridad, fecha_creacion, fecha_resolucion, notas_resolucion
- usuarios: id_usuario (PK), id_empleado (FK), id_cliente (FK), username, password, nombre, email, rol, activo, fecha_creacion
- kb_articulos: id_articulo (PK), titulo, problema, solucion, keywords, veces_util, fecha_creacion

Relaciones Cardinales.

departamentos — empleados (1:N) — Un departamento tiene varios empleados
empleados — usuarios (1:1) — Cada empleado tiene un usuario web
clientes — usuarios (1:1) — Cada cliente tiene un usuario web
clientes — contratos (1:N) — Un cliente puede tener varios contratos
servicios — contratos (1:N) — Un servicio puede estar en varios contratos
clientes — incidencias (1:N) — Un cliente puede tener varias incidencias

empleados — incidencias (1:N) — Un empleado puede tener varias incidencias asignadas

6.4 Modelo Entidad-Relación

```
echo "==========================================="
echo "Configurando Base de Datos..."
echo "==========================================="

apt-get update
apt-get install -y mariadb-server mariadb-client

# Escuchar en todas las interfaces
sed -i 's/^bind-address.*/bind-address = 0.0.0.0/'
/etc/mysql/mariadb.conf.d/50-server.cnf
systemctl restart mariadb

# Crear base de datos y estructura
mysql -u root <<'EOSQL'
CREATE DATABASE IF NOT EXISTS verificanet_servicios;
USE verificanet_servicios;

CREATE TABLE IF NOT EXISTS departamentos (
    id_departamento INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT
);

CREATE TABLE IF NOT EXISTS empleados (
    id_empleado INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    id_departamento INT,
    fecha_contratacion DATE,
    FOREIGN KEY (id_departamento) REFERENCES
departamentos(id_departamento)
);

CREATE TABLE IF NOT EXISTS clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
```

```
    apellido VARCHAR(100),
    nombre_empresa VARCHAR(200),
    email VARCHAR(100) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    direccion TEXT,
    tipo_cliente ENUM('particular', 'empresa') DEFAULT 'particular',
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS servicios (
    id_servicio INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    precio_base DECIMAL(10,2),
    tipo_servicio ENUM('soporte', 'mantenimiento', 'desarrollo',
'consultoria') DEFAULT 'soporte'
);

CREATE TABLE IF NOT EXISTS contratos (
    id_contrato INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    id_servicio INT,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    estado ENUM('activo', 'inactivo', 'finalizado') DEFAULT 'activo',
    notas TEXT,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE
CASCADE,
    FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
);

CREATE TABLE IF NOT EXISTS incidencias (
    id_incidencia INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    descripcion TEXT,
    estado ENUM('reportada', 'en_proceso', 'resuelta', 'cerrada')
DEFAULT 'reportada',
    prioridad ENUM('baja', 'media', 'alta', 'critica') DEFAULT 'media',
    id_cliente INT,
    id_empleado_asignado INT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_resolucion DATETIME,
```

```
    notas_resolucion TEXT,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE
CASCADE,
    FOREIGN KEY (id_empleado_asignado) REFERENCES
empleados(id_empleado)
);

CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    nombre VARCHAR(100),
    email VARCHAR(100),
    rol ENUM('admin', 'empleado', 'tecnico', 'cliente') NOT NULL,
    id_empleado INT NULL,
    id_cliente INT NULL,
    activo BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado) ON
DELETE CASCADE,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE
CASCADE
);

CREATE TABLE IF NOT EXISTS kb_articulos (
    id_articulo INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(200) NOT NULL,
    problema TEXT,
    solucion TEXT,
    keywords VARCHAR(500),
    veces_util INT DEFAULT 0,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- DEPARTAMENTOS
INSERT INTO departamentos (nombre, descripcion) VALUES
('Soporte Técnico', 'Atención al cliente y soporte técnico'),
('Ventas', 'Equipo comercial y ventas'),
('Administración', 'Gestión administrativa'),
('Sistemas', 'Administración de sistemas y desarrollo'),
('Auditoría', 'Auditoría y seguridad');
```

```
-- EMPLEADOS
INSERT INTO empleados (nombre, apellido, email, telefono,
id_departamento, fecha_contratacion) VALUES
('María', 'González', 'mgonzalez@verificanet.local', '666111222', 1,
'2022-01-15'),
('Juan', 'Martínez', 'jmartinez@verificanet.local', '666222333', 1,
'2022-03-20'),
('Laura', 'Fernández', 'lfernandez@verificanet.local', '666555666', 2,
'2022-06-15'),
('Sofía', 'Torres', 'storres@verificanet.local', '666777888', 2,
'2023-01-10'),
('Ana', 'Rodríguez', 'arodriguez@verificanet.local', '666333444', 3,
'2021-11-10'),
('Miguel', 'Navarro', 'mnavarro@verificanet.local', '666888999', 3,
'2023-03-15'),
('Carlos', 'López', 'clopez@verificanet.local', '666444555', 4,
'2023-02-01'),
('Admin', 'VerificaNet', 'adminvnet@verificanet.local', '666999000', 4,
'2021-01-01'),
('Pedro', 'Sánchez', 'psanchez@verificanet.local', '666666777', 5,
'2021-08-20');

-- CLIENTES
INSERT INTO clientes (nombre, apellido, email, telefono, tipo_cliente)
VALUES
('Particular', 'Test', 'cliente1@example.com', '611222333',
'particular'),
(NULL, NULL, 'contacto@empresa.com', '622333444', 'empresa');
UPDATE clientes SET nombre_empresa = 'Empresa Test S.L.' WHERE email =
'contacto@empresa.com';

-- SERVICIOS
INSERT INTO servicios (nombre, descripcion, precio_base, tipo_servicio)
VALUES
('Soporte Técnico Básico', 'Asistencia técnica remota 24/7', 299.99,
'soporte'),
('Mantenimiento Preventivo', 'Mantenimiento mensual de sistemas',
499.99, 'mantenimiento'),
('Desarrollo Web', 'Desarrollo de aplicaciones web a medida', 2999.99,
'desarrollo'),
('Consultoría IT', 'Consultoría en infraestructura y seguridad',
1499.99, 'consultoria'),
```

```
('Auditoría de Seguridad', 'Análisis completo de seguridad
informática', 1999.99, 'consultoria'),
('Backup Cloud', 'Servicio de copias de seguridad en la nube', 199.99,
'soporte');

-- CONTRATOS
INSERT INTO contratos (id_cliente, id_servicio, fecha_inicio, estado)
VALUES
(1, 1, '2024-01-01', 'activo'),
(1, 6, '2024-01-01', 'activo'),
(2, 2, '2024-02-01', 'activo'),
(2, 3, '2024-02-15', 'activo'),
(2, 5, '2024-03-01', 'activo');

-- INCIDENCIAS
INSERT INTO incidencias (titulo, descripcion, estado, prioridad,
id_cliente, id_empleado_asignado) VALUES
('Error en conexión VPN', 'No se puede conectar a la VPN corporativa',
'en_proceso', 'alta', 2, 1),
('Lentitud en servidor', 'El servidor web responde muy lento',
'reportada', 'media', 2, 2),
('Backup fallido', 'El backup automático no se ejecutó anoche',
'resuelta', 'alta', 1, 1),
('Solicitud nueva funcionalidad', 'Añadir módulo de reportes en la
aplicación', 'reportada', 'baja', 2, 7),
('Problema con correo', 'No recibo correos desde esta mañana',
'cerrada', 'critica', 1, 1),
('Actualización sistema', 'Necesito actualizar el sistema operativo',
'en_proceso', 'media', 2, 2);

-- USUARIOS con contraseñas hasheadas (hash de Verific@2024!)
INSERT INTO usuarios (username, password, nombre, email, rol,
id_empleado) VALUES
('adminvnet',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.',
'Administrador', 'admin@verificanet.com', 'admin', 8),

('mgonzalez',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'María
González', 'mgonzalez@verificanet.com', 'empleado', 1),
```

```
('jmartinez',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'Juan
Martínez', 'jmartinez@verificanet.com', 'empleado', 2),

('lfernandez',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'Laura
Fernández', 'lfernandez@verificanet.com', 'empleado', 3),
('storres',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'Sofía
Torres', 'storres@verificanet.com', 'empleado', 4),

('arodriguez',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'Ana
Rodríguez', 'arodriguez@verificanet.com', 'empleado', 5),

('mnavarro',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.', 'Miguel
Navarro', 'mnavarro@verificanet.com', 'empleado', 6),


INSERT INTO usuarios (username, password, nombre, email, rol,
id_cliente) VALUES
('cliente1',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.',
'Cliente Demo', 'cliente1@example.com', 'cliente', 1),

('empresa1',
'$2y$10$Fmhe0Rf3zQIfTKpMNtGktO4m7RSydcggVxEFkAktULo9mnoZxVAr.',
'Empresa Demo', 'contacto@empresa.com', 'cliente', 2);

-- KB ARTÍCULOS
INSERT INTO kb_articulos (titulo, problema, solucion, keywords,
veces_util) VALUES
('No puedo conectarme a la VPN', 'El cliente no puede conectarse a la
VPN corporativa', 'Verificar credenciales y que el cliente VPN esté
actualizado', 'vpn conexion red', 15),
('Correo no llega', 'No se reciben correos desde esta mañana', 'Revisar
carpeta spam y configuración del servidor SMTP', 'correo email smtp',
10),
('Lentitud en el sistema', 'El sistema va muy lento', 'Limpiar caché
del navegador y reiniciar el equipo', 'lento rendimiento velocidad',
8);
```

6.5 Permisos de Usuario

El usuario verificanet_user tiene permisos sobre la base de datos desde las IPs de los backends y el Windows Server:

```
-- Crear usuario para cada IP permitida
CREATE USER 'verificanet_user'@'192.168.50.41' IDENTIFIED BY 'Verific@2024!';
CREATE USER 'verificanet_user'@'192.168.50.42' IDENTIFIED BY 'Verific@2024!';
CREATE USER 'verificanet_user'@'192.168.50.10' IDENTIFIED BY 'Verific@2024!';
CREATE USER 'verificanet_user'@'192.168.60.50' IDENTIFIED BY 'Verific@2024!';
CREATE USER 'verificanet_user'@'localhost'     IDENTIFIED BY 'Verific@2024!';

-- Otorgar permisos sobre la base de datos
GRANT SELECT, INSERT, UPDATE, DELETE ON verificanet_servicios.*
    TO 'verificanet_user'@'192.168.50.41';
GRANT SELECT, INSERT, UPDATE, DELETE ON verificanet_servicios.*
    TO 'verificanet_user'@'192.168.50.42';
-- (repetir para cada usuario)

FLUSH PRIVILEGES;
```

7. Aplicación Web

La aplicación web está desarrollada en PHP y se sirve desde dos backends Apache balanceados por Nginx. Implementa autenticación basada en sesiones PHP con redirección automática según el rol y departamento del usuario.

7.1 Configuración de Nginx (Balanceador)

Nginx actúa como punto de entrada único en el puerto 80. Usa ip_hash para que cada usuario siempre vaya al mismo backend, garantizando que las sesiones PHP no se pierdan:

```
upstream backends {
    ip_hash;
    server 192.168.50.41;
    server 192.168.50.42;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backends;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }
}
```

7.2 Sistema de Autenticación

El proceso de login consulta la tabla usuarios junto con la tabla departamentos para determinar a qué página redirigir. Las contraseñas se almacenan con bcrypt:

```
<?php
// Consulta que obtiene usuario + departamento en un solo JOIN
$stmt = $conn->prepare('
    SELECT u.id_usuario, u.nombre, u.password, u.rol,
           d.nombre as departamento
    FROM usuarios u
    LEFT JOIN empleados e ON u.id_empleado = e.id_empleado
    LEFT JOIN departamentos d ON e.id_departamento = d.id_departamento
    WHERE u.username = ?
');

// Verificar contraseña con bcrypt
if (password_verify($password, $user['password'])) {
    $_SESSION['usuario_id']     = $user['id_usuario'];
    $_SESSION['usuario_nombre'] = $user['nombre'];
    $_SESSION['rol']            = $user['rol'];
    $_SESSION['departamento']   = $user['departamento'];

    // Redirigir según rol y departamento
    switch ($user['rol']) {
        case 'admin':   redirigir('../admin.php'); break;
        case 'tecnico': redirigir('../tecnico.php'); break;
        case 'cliente': redirigir('../servicios.php'); break;
        case 'empleado':
            switch ($user['departamento']) {
                case 'Ventas':        redirigir('../ventas.php'); break;
                case 'Administración':redirigir('../administracion.php'); break;
                case 'Auditoría':     redirigir('../auditoria.php'); break;
                default:              redirigir('../empleado.php');
            }
    }
}
```

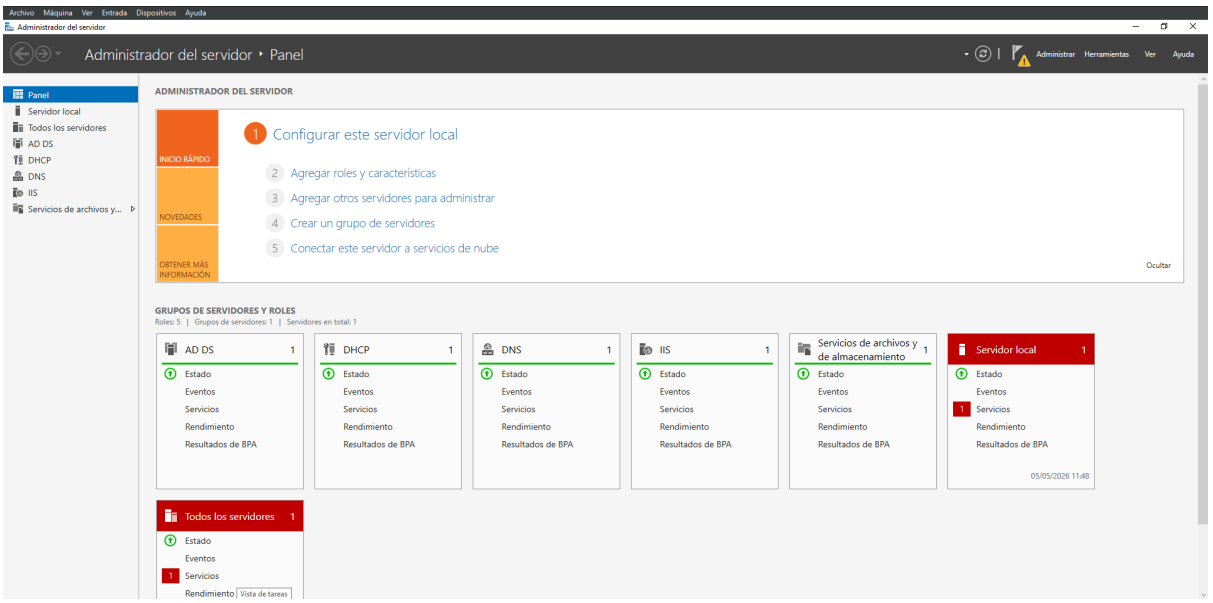
7.3 Conexión a la Base de Datos

El archivo includes/config.php centraliza la conexión a MariaDB y las funciones de autenticación usadas por todas las páginas:

```php
<?php
session_start();

define('DB_HOST', '192.168.60.50');
define('DB_NAME', 'verificanet_servicios');
define('DB_USER', 'verificanet_user');
define('DB_PASS', 'Verific@2024!');

function conectar_bd() {
    $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);
    if ($conn->connect_error) {
        die('Error de conexion: ' . $conn->connect_error);
    }
    $conn->set_charset('utf8');
    return $conn;
}

function requiere_rol($rol) {
    if (!isset($_SESSION['rol']) || $_SESSION['rol'] !== $rol) {
        header('Location: login.php');
        exit();
    }
}
```

8. Windows Server — Active Directory

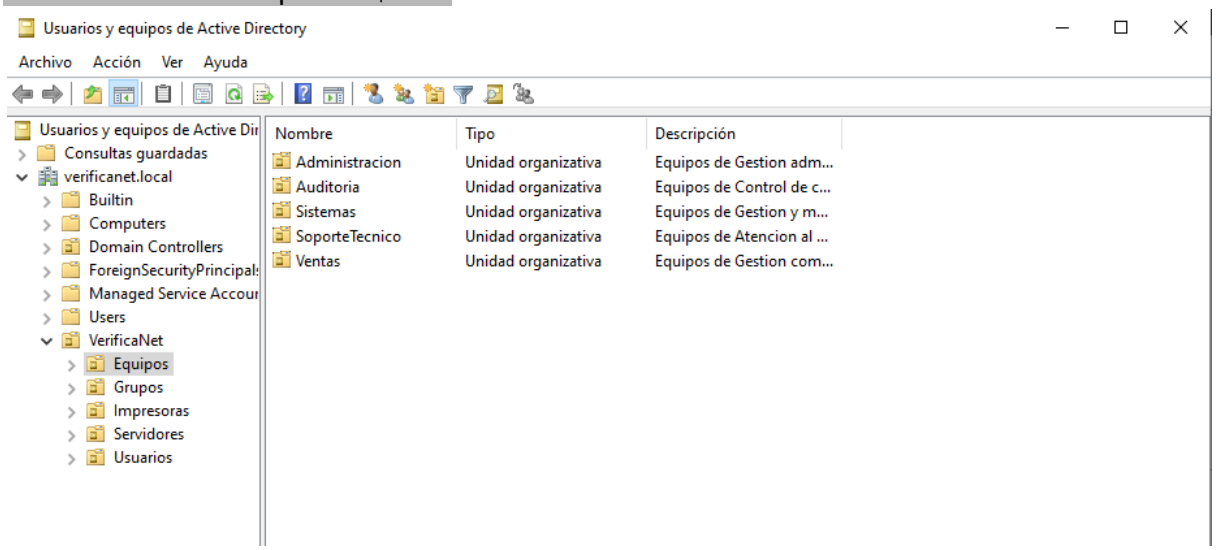
El Windows Server 2019 (192.168.50.10) actúa como controlador de dominio principal del dominio verificanet.local. Proporciona gestión centralizada de usuarios, grupos y políticas de seguridad mediante Active Directory Domain Services (AD DS).



8.1 Instalación y Configuración del Dominio

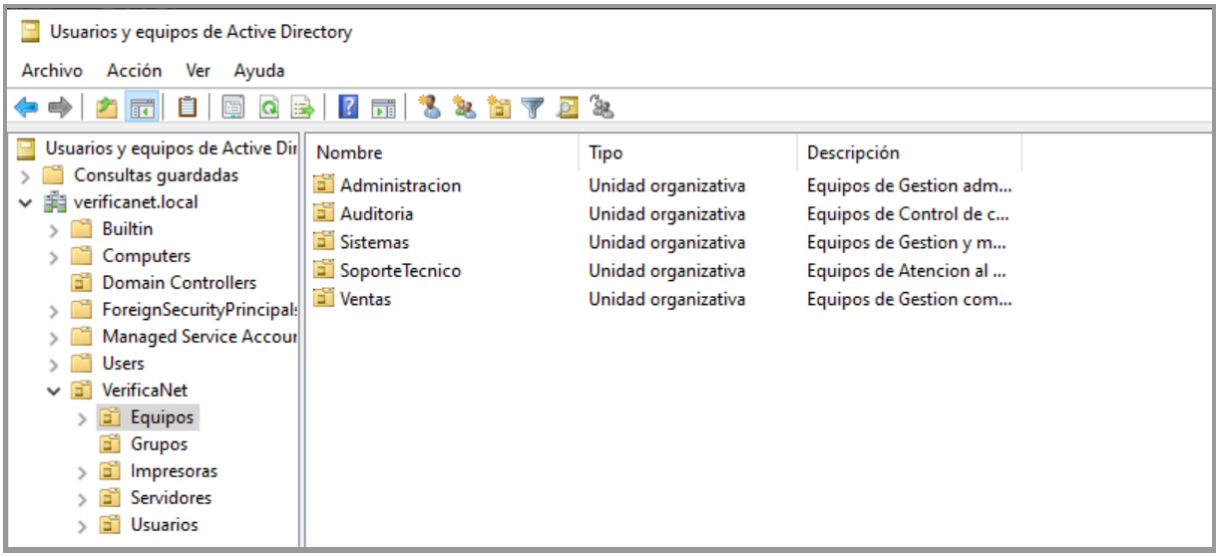
El script Active_Directory.ps1 automatiza la instalación del rol AD DS y la promoción del servidor a Domain Controller:

```
# Configuracion del dominio
$dominio     = 'verificanet.local'
$netbios     = 'VERIFICANET'
$passwordDSRM = 'Admin123!'

# Paso 1: Instalar el rol AD Domain Services
Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools

# Paso 2: Promover el servidor a Domain Controller
$safePwd = ConvertTo-SecureString $passwordDSRM -AsPlainText -Force

Install-ADDSForest `
    -DomainName $dominio `
    -DomainNetbiosName $netbios `
    -ForestMode 'WinThreshold' `
    -DomainMode 'WinThreshold' `
    -InstallDns:$true `
    -SafeModeAdministratorPassword $safePwd `
    -Force:$true `
    -NoRebootOnCompletion:$false
```



8.2 Unidades Organizativas

El script OUS.ps1 crea la estructura de Unidades Organizativas que refleja la jerarquía de la empresa:



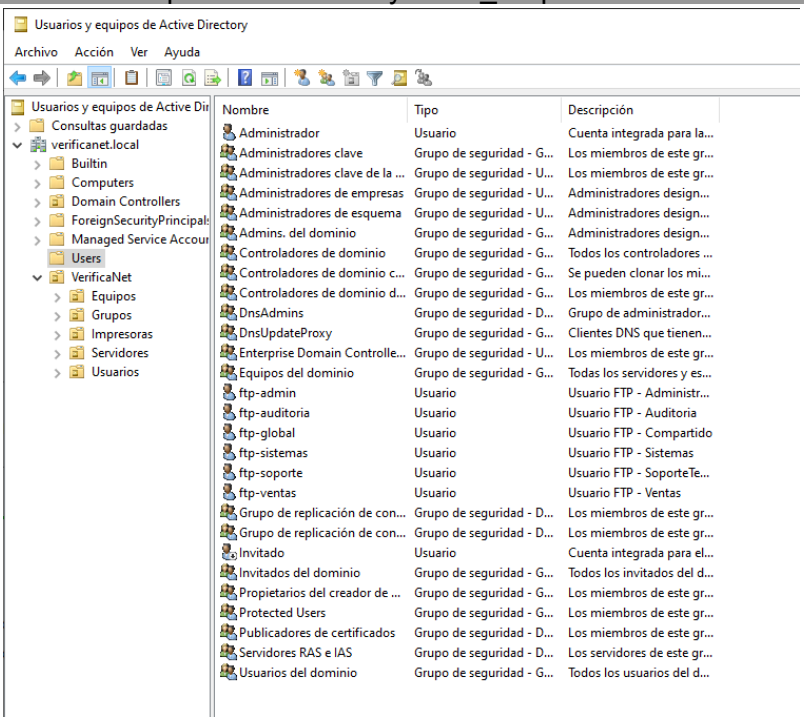

```
# Crear Unidades Organizativas (OUs) en el dominio
$dominioDN = 'DC=verificanet,DC=local'

New-ADOrganizationalUnit -Name 'Empleados'     -Path $dominioDN
New-ADOrganizationalUnit -Name 'Clientes'      -Path $dominioDN
New-ADOrganizationalUnit -Name 'Administracion'-Path $dominioDN
New-ADOrganizationalUnit -Name 'Sistemas'      -Path $dominioDN
New-ADOrganizationalUnit -Name 'Auditoria'     -Path $dominioDN

# Crear grupos de seguridad por departamento
New-ADGroup -Name 'GRP_Empleados'  -GroupScope Global `
   -Path 'OU=Empleados,DC=verificanet,DC=local'
New-ADGroup -Name 'GRP_Sistemas'   -GroupScope Global `
   -Path 'OU=Sistemas,DC=verificanet,DC=local'
New-ADGroup -Name 'GRP_Auditoria'  -GroupScope Global `
   -Path 'OU=Auditoria,DC=verificanet,DC=local'
```

8.3 Creación de Usuarios en AD

El script Usuarios-grupos.ps1 crea los usuarios del dominio y los asigna a sus grupos y OUs correspondientes:

```
# Ejemplo de creacion de usuario en Active Directory
$password = ConvertTo-SecureString 'Verific@2024!' -AsPlainText -Force

New-ADUser `
   -Name            'Maria Gonzalez' `
   -SamAccountName  'mgonzalez' `
   -UserPrincipalName 'mgonzalez@verificanet.local' `
   -Path            'OU=Empleados,DC=verificanet,DC=local' `
```

```
    -AccountPassword $password `
    -Enabled         $true `
    -Department      'Soporte Tecnico' `
    -Title           'Tecnico de Soporte'

# Añadir al grupo correspondiente
Add-ADGroupMember -Identity 'GRP_Empleados' -Members 'mgonzalez'
```



9. Windows Server — DNS

El servicio DNS está integrado con Active Directory y proporciona resolución de nombres para todos los equipos del dominio verificanet.local. El script DNS.ps1 automatiza su configuración completa.

9.1 Instalación y Zonas DNS

```
# Variables de configuracion
$dominio     = 'verificanet.local'
$zonaInversa = '50.168.192.in-addr.arpa'
$servidorIP  = '192.168.50.10'

# Instalar rol DNS
Install-WindowsFeature -Name DNS -IncludeManagementTools

# Crear zona de busqueda directa
Add-DnsServerPrimaryZone `
    -Name $dominio `
    -ReplicationScope 'Forest' `
    -DynamicUpdate 'Secure'
```

```
# Crear zona de busqueda inversa
Add-DnsServerPrimaryZone `
    -NetworkID '192.168.50.0/24' `
    -ReplicationScope 'Forest' `
    -DynamicUpdate 'Secure'
```

9.2 Registros DNS

Se crean registros A con su correspondiente PTR inverso para cada servidor de la infraestructura:

```
# Registros A para cada servidor
$registrosA = @(
    @{Nombre='srv-win';      IP='192.168.50.10'},
    @{Nombre='srv-web';      IP='192.168.50.30'},
    @{Nombre='srv-backend1'; IP='192.168.50.41'},
    @{Nombre='srv-backend2'; IP='192.168.50.42'},
    @{Nombre='srv-firewall'; IP='192.168.50.1'}
)

foreach ($r in $registrosA) {
    Add-DnsServerResourceRecordA `
        -ZoneName $dominio `
        -Name $r.Nombre `
        -IPv4Address $r.IP `
        -CreatePtr   # Crea el registro PTR inverso automaticamente
}

# Registros CNAME (alias)
Add-DnsServerResourceRecordCName `
    -ZoneName $dominio -Name 'www' `
    -HostNameAlias 'srv-web.verificanet.local.'
Add-DnsServerResourceRecordCName `
    -ZoneName $dominio -Name 'ftp' `
    -HostNameAlias 'srv-win.verificanet.local.'
```

9.3 Registros Creados

Nombre DNS	Tipo	Valor
srv-win.verificanet.local	A	192.168.50.10
srv-web.verificanet.local	A	192.168.50.30
srv-backend1.verificanet.local	A	192.168.50.41
srv-backend2.verificanet.local	A	192.168.50.42
srv-firewall.verificanet.local	A	192.168.50.1

www.verificanet.local	CNAME	srv-web.verificanet.local
ftp.verificanet.local	CNAME	srv-win.verificanet.local

10. Windows Server — DHCP

El servicio DHCP asigna direcciones IP dinámicamente a los equipos de la red DMZ. Está autorizado en Active Directory y tiene configuradas reservas fijas para todos los servidores de la infraestructura.

10.1 Configuración del Scope DHCP

```
# Variables DHCP
$servidorIP   = '192.168.50.10'
$puertaEnlace = '192.168.50.1'
$rangoInicio  = '192.168.50.100'
$rangoFin     = '192.168.50.200'
$mascara      = '255.255.255.0'
$dominio      = 'verificanet.local'

# Instalar y autorizar DHCP en AD
Install-WindowsFeature -Name DHCP -IncludeManagementTools
Add-DhcpServerInDC -DnsName 'WIN-SERVER.verificanet.local' `
                   -IPAddress $servidorIP

# Crear el scope (ambito)
Add-DhcpServerv4Scope `
   -Name       'LAN_VerificaNet' `
   -StartRange $rangoInicio `
   -EndRange   $rangoFin `
   -SubnetMask $mascara `
   -State      Active

# Configurar opciones (gateway, DNS, dominio)
$scopeID = (Get-DhcpServerv4Scope).ScopeId
Set-DhcpServerv4OptionValue -ScopeId $scopeID `
   -Router    $puertaEnlace `
   -DnsServer $servidorIP `
   -DnsDomain $dominio

# Excluir IPs de servidores del rango dinamico
Add-DhcpServerv4ExclusionRange -ScopeId $scopeID `
   -StartRange '192.168.50.1' -EndRange '192.168.50.50'
```

10.2 Reservas para Servidores

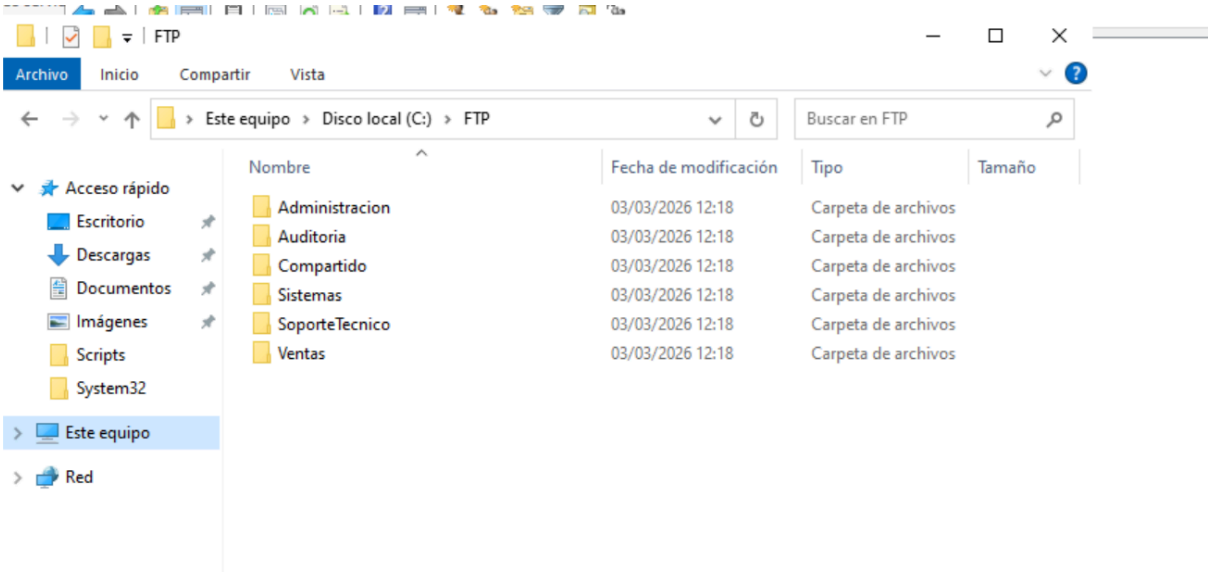
Se crean reservas DHCP para que los servidores Vagrant siempre reciban la misma IP basándose en su dirección MAC:

```
# Reservas para servidores con IP fija
$reservas = @(
```

```
    @{Nombre='SRV-FIREWALL'; IP='192.168.50.1';  MAC='08-00-27-26-60-63'},
    @{Nombre='SRV-WEB';      IP='192.168.50.30'; MAC='08-00-27-93-03-24'},
    @{Nombre='SRV-BACKEND1'; IP='192.168.50.41'; MAC='08-00-27-3b-60-14'},
    @{Nombre='SRV-BACKEND2'; IP='192.168.50.42'; MAC='08-00-27-d8-24-a7'}
)

foreach ($r in $reservas) {
    Add-DhcpServerv4Reservation `
        -ScopeId    $scopeID `
        -IPAddress  $r.IP `
        -ClientId   $r.MAC `
        -Name       $r.Nombre
}

# Reiniciar servicio DHCP
Restart-Service -Name DHCPServer
```

11. Windows Server — FTP

El servicio FTP está configurado en el Windows Server para proporcionar un punto de transferencia de archivos para la empresa. Está integrado con IIS (Internet Information Services) y con Active Directory para la autenticación de usuarios.



11.1 Instalación del Servicio FTP

```
# Instalar IIS con soporte FTP
Install-WindowsFeature -Name Web-Server, Web-Ftp-Server `
                       -IncludeManagementTools

# Crear directorio raiz del FTP
New-Item -Path 'C:\FTP' -ItemType Directory -Force

# Crear sitio FTP en IIS
New-WebFtpSite -Name 'VerificaNet-FTP' `
```

```
            -Port 21 `
            -PhysicalPath 'C:\FTP'

# Configurar autenticacion basica
Set-WebConfigurationProperty `
   -Filter '/system.ftpServer/security/authentication/basicAuthentication' `
   -PSPath  'IIS:\Sites\VerificaNet-FTP' `
   -Name    'enabled' -Value $true

# Activar el firewall de Windows para FTP
New-NetFirewallRule -DisplayName 'FTP' `
   -Direction Inbound -Protocol TCP -LocalPort 21 -Action Allow
```

12. Sistema de Monitorización (Ampliación)

Verificanet incluye un sistema de monitorización de infraestructura accesible desde la web para los administradores. Permite comprobar el estado de todos los servidores y reiniciar servicios directamente desde el navegador.

12.1 Arquitectura del Sistema

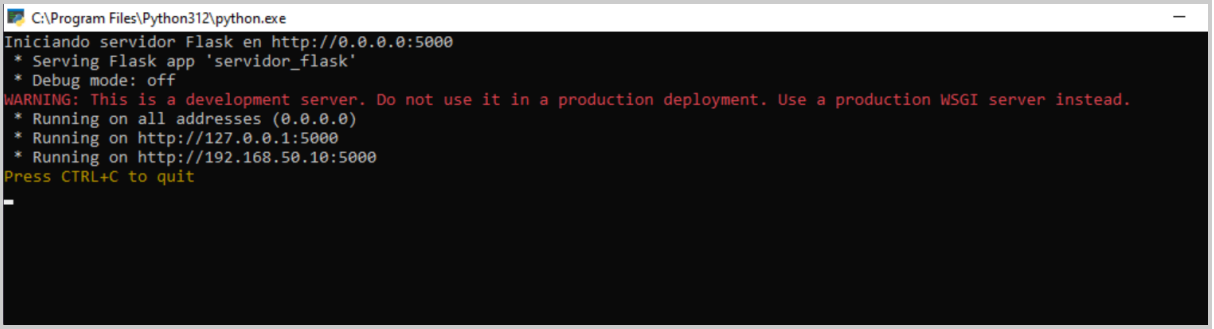
El sistema funciona en tres capas independientes:

7. estado_sistema.php — Interfaz web con tarjetas por servidor. Envía peticiones HTTP al Flask.
8. servidor_flask.py — API REST en Python ejecutándose en Windows Server en el puerto 5000. Recibe peticiones HTTP y ejecuta el script Python.
9. admin_servidor.py — Script Python que hace SSH a cada servidor Linux y ejecuta los comandos de consulta.

El flujo completo de una consulta es: Navegador → PHP → HTTP → Flask (Windows:5000) → SSH → Servidor Linux → respuesta a la inversa.

12.2 Servidor Flask (servidor_flask.py)

El servidor Flask actúa como API REST intermediaria. Recibe peticiones HTTP del PHP, ejecuta el script Python y devuelve el resultado en formato JSON:



```
from flask import Flask, request, jsonify
import subprocess

app = Flask(__name__)

PYTHON = r'C:\Program Files\Python312\python.exe'
SCRIPT = r'C:\Scripts\admin_servidor.py'

ACCIONES_PERMITIDAS = [
    'estado_nginx', 'estado_apache', 'estado_mariadb',
    'syslog', 'reiniciar_nginx', 'reiniciar_apache',
    'reiniciar_mariadb', 'usuarios_bd', 'incidencias_bd',
]

@app.route('/estado', methods=['GET'])
def estado():
    accion   = request.args.get('accion', '')
    servidor = request.args.get('servidor', '')

    resultado = subprocess.run(
        [PYTHON, SCRIPT, accion, servidor],
        capture_output=True, timeout=20,
        stdin=subprocess.DEVNULL
    )
    salida = resultado.stdout.decode('utf-8', errors='replace')
    return jsonify({'resultado': salida.strip()})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

12.3 Script de Consulta (admin_servidor.py)

El script admin_servidor.py hace SSH a cada servidor Linux y ejecuta el comando correspondiente. Usa la clave privada del Administrador para autenticarse sin contraseña:

```
import sys, subprocess

USUARIO  = 'vagrant'
SSH_PATH = r'C:\Windows\System32\OpenSSH\ssh.exe'
CLAVE    = r'C:\Users\Administrador\.ssh\id_rsa'

SERVIDORES = {
    'web':      '192.168.50.30',
    'backend1': '192.168.50.41',
    'backend2': '192.168.50.42',
    'database': '192.168.60.50',
    'firewall': '192.168.50.20',
}

comandos = {
    'estado_apache':  'systemctl --no-pager status apache2 2>&1',
```

```
    'estado_nginx':   'systemctl --no-pager status nginx 2>&1',
    'estado_mariadb': 'systemctl --no-pager status mariadb 2>&1',
    'syslog':         'sudo tail -n 20 /var/log/syslog 2>&1',
    'usuarios_bd':    'mysql -h 127.0.0.1 -u verificanet_user ...',
}

def ejecutar_ssh(ip, comando):
    args = [SSH_PATH, '-n', '-i', CLAVE,
            '-o', 'StrictHostKeyChecking=no',
            '-o', 'BatchMode=yes',
            f'{USUARIO}@{ip}', comando]
    resultado = subprocess.run(args, capture_output=True,
                               timeout=15, stdin=subprocess.DEVNULL)
    return resultado.stdout.decode('utf-8', errors='replace')
```

12.4 Consulta desde PHP

El archivo estado_sistema.php realiza una petición HTTP al Flask usando file_get_contents y muestra el resultado en una caja de código en la página:

```
<?php
function consultar_flask($accion, $servidor) {
    $url = 'http://192.168.50.10:5000/estado'
         . '?accion='   . urlencode($accion)
         . '&servidor=' . urlencode($servidor);

    $ctx = stream_context_create([
        'http' => ['timeout' => 25, 'method' => 'GET']
    ]);
    $respuesta = @file_get_contents($url, false, $ctx);

    if ($respuesta === false) {
        return 'Error: No se pudo conectar al Flask';
    }
    $json = json_decode($respuesta, true);
    return $json['resultado'] ?? 'Sin respuesta';
}
```

12.5 Arranque Automático del Flask

El Flask está configurado como tarea programada en Windows para arrancar automáticamente con el sistema, ejecutándose con el usuario Administrador:

```
# Crear tarea programada para Flask
$action   = New-ScheduledTaskAction `
    -Execute  'C:\Program Files\Python312\python.exe' `
    -Argument 'C:\Scripts\servidor_flask.py'

$trigger  = New-ScheduledTaskTrigger -AtStartup
```

```
$settings = New-ScheduledTaskSettingsSet `
    -RestartCount 3 `
    -RestartInterval (New-TimeSpan -Minutes 1)

Register-ScheduledTask `
    -TaskName 'FlaskVerificanet' `
    -Action   $action `
    -Trigger  $trigger `
    -RunLevel Highest `
    -User     'VERIFICANET\Administrador' `
    -Settings $settings -Force

# Verificar que Flask esta activo
netstat -an | findstr 5000
# Debe aparecer: TCP  0.0.0.0:5000  LISTENING
```

13. Usuarios y Accesos

13.1 Usuarios del Sistema Web

Todos los usuarios tienen la contraseña: Verific@2024! (almacenada como hash scrypt en la base de datos)

Usuario	Nombre	Rol	Departamento / Acceso
adminvnet	Administrador	admin	admin.php + estado_sistema.php
clopez	Carlos López	tecnico	tecnico.php
mgonzalez	María González	empleado	Soporte → empleado.php
jmartinez	Juan Martínez	empleado	Soporte → empleado.php
lfernandez	Laura Fernández	empleado	Ventas → ventas.php
storres	Sofía Torres	empleado	Ventas → ventas.php
arodriguez	Ana Rodríguez	empleado	Administración → administracion.php
mnavarro	Miguel Navarro	empleado	Administración → administracion.php
psanchez	Pedro Sánchez	empleado	Auditoría → auditoria.php
cliente1	Cliente Demo	cliente	servicios.php
empresa1	Empresa Demo	cliente	servicios.php

13.2 Acceso a la Aplicación

La aplicación web es accesible desde el navegador del host con las máquinas virtuales levantadas:

```
# URL de acceso a la aplicacion web
http://localhost:8080

#URL de acceso a la aplicación web desde clientes
http://www.verificanet.local

# Para levantar la infraestructura completa
vagrant up router firewall database backend1 backend2 web

# Para comprobar el estado de las VMs
vagrant status
```

14. Solución de Errores

1. Connection timed out en config.php
   - Causa: Las rutas entre backends y base de datos se perdían al reiniciar las VMs
   - Solución: Configurar rutas estáticas permanentes con Netplan (via 192.168.50.20) y ajustar el bind-address de MariaDB a 0.0.0.0

2. 504 Gateway Time-out en nginx
   - Causa: Los backends tardaban demasiado en responder o Apache tenía procesos colgados
   - Solución: Reiniciar Apache (systemctl restart apache2) y ajustar proxy_read_timeout 60s en nginx

3. Permission denied (publickey) en SSH
   - Causa: La clave pública no estaba en el lugar correcto para el usuario Administrador en Windows
   - Solución: Copiar la clave a C:\ProgramData\ssh\administrators_authorized_keys y dar permisos con icacls

4. Unknown column 'id_empleado' en empleado.php
   - Causa: La columna en la BD se llama id_empleado_asignado pero el código usaba id_empleado
   - Solución: sed -i 's/WHERE id_empleado = ?/WHERE id_empleado_asignado = ?/g'

5. index.html sobreescribía index.php
   - Causa: Apache da prioridad a index.html sobre index.php

- Solución: Eliminar el index.html del directorio web y añadir rm -f /var/www/html/index.html al backend.sh

6. Flask no arrancaba al iniciar Windows
   - Causa: La tarea programada usaba SYSTEM y Python no estaba en su PATH
   - Solución: Crear la tarea con usuario Administrador y trigger AtLogOn

7. "C:\Program" no se reconoce como comando
   - Causa: Los espacios en C:\Program Files\Python312\python.exe rompían el comando SSH
   - Solución: Crear C:\Scripts\ejecutar.bat que llama a Python internamente

8. Contraseñas no verificaban en login
   - Causa: El database.sh guardaba las contraseñas en texto plano pero PHP usa password_verify()
   - Solución: Generar hash bcrypt con password_hash() y actualizar la BD con UPDATE usuarios SET password = '$2y$10$...'

9. Sesiones perdidas entre backends
   - Causa: Nginx enviaba peticiones a backends distintos y la sesión PHP no se compartía
   - Solución: Añadir ip_hash al bloque upstream de nginx

10. Puerto 5000 bloqueado
    - Causa: El firewall de Windows bloqueaba conexiones entrantes al puerto de Flask
    - Solución: New-NetFirewallRule -DisplayName "Flask-5000" -Direction Inbound -Protocol TCP -LocalPort 5000 -Action Allow

15. Herramientas Utilizadas

En este Proyecto he podido utilizar varias herramientas, que me han aportado facilidades y oportunidades para desarrollar esta ampliación.

Claude.ia -> Esta herramienta me ha ayudado a solucionar problemas tanto en la configuración de mis máquinas linux como en el desarrollo de mis páginas webs.

https://learn.microsoft.com/es-es/windows-server/ -> La documentación de microsoft me ha sido muy útil debido a que la mayoría de errores se han podido solucionar ciertos errores.

Synapse.es ->Gracias al equipo de trabajo de la empresa donde he realizado mis prácticas he podido solucionar muchos errores de diseño y maquetación de la web además de ayudarme con la conexión entre windows y linux mediante ssh.

16. Conclusiones

El proyecto Verificanet ha cumplido con todos los objetivos planteados al inicio. Se ha conseguido desplegar una infraestructura empresarial completa y funcional que integra múltiples tecnologías del ámbito de la administración de sistemas.

Los principales logros del proyecto son:

- Infraestructura segmentada con DMZ y red interna protegida por un firewall con reglas iptables específicas para cada tipo de tráfico.
- Despliegue completamente automatizado mediante Vagrant y scripts de aprovisionamiento bash y PowerShell, lo que permite recrear toda la infraestructura de cero con un solo comando.
- Aplicación web PHP funcional con 7 roles de usuario, dashboards diferenciados por departamento y sistema de incidencias completo.
- Alta disponibilidad en la capa web mediante dos backends Apache balanceados con Nginx usando ip_hash para mantener las sesiones.
- Windows Server con Active Directory, DNS con zonas directas e inversas, DHCP con reservas por MAC, FTP y OpenSSH integrados.
- Sistema de monitorización en tiempo real que combina PHP, Flask, Python y SSH en un flujo de tres capas para consultar el estado de todos los servidores desde el navegador.
- Integración de Python con la base de datos para ejecutar consultas SQL remotas desde la interfaz web de administración.

El proyecto ha supuesto un reto técnico muy importante, especialmente en la configuración del firewall, la comunicación entre redes y el sistema de monitorización con el paso intermedio por el Windows Server. Estos obstáculos han permitido profundizar en el conocimiento de iptables, routing entre redes, SSH, APIs REST con Flask y la integración de sistemas con diferentes Sistemas Operativos (Linux + Windows).

Verificanet representa un ejemplo real de cómo se despliega y gestiona una infraestructura empresarial completa, integrando todos los conocimientos adquiridos durante el Ciclo Formativo de Grado Superior de ASIR.

Además de una gran ampliación de conocimientos en el plazo de prácticas el cual he podido aprender la implementación más a fondo de php y la integración de python con Windows.

Pudiendo realizar la ampliación y modificando muchas partes del proyecto.